

COURSE CODE/TITLE:	CPE212/Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	9/2/2026	DATE SUBMITTED:	9/2/2026
NAME:	Javillonar, Madhel Kathlyn Ann A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 Targeting Specific Nodes

1. PROCEDURE OUTPUT

Requirement:

In this activity, you will need to create another Ubuntu VM and name it Server 3. Likewise, you need to activate the second adapter to a host-only adapter after the installations. Take note of the IP address of the Server 3. Make sure to use the command `ssh-copy-id` to copy the public key to Server 3. Verify if you can successfully SSH to Server 3.

Task 1: Targeting Specific Nodes

1. Create a new playbook and named it `site.yml`. Follow the commands as shown

```
---
- hosts: all
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
      when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    dnf:
      name:
        - httpd
        - php
      state: latest
      when: ansible_distribution == "CentOS"
```

NOTE:

Since the Ubuntu servers and the CentOS server use different sudo passwords, running the playbook with the Ubuntu sudo password allows the Ubuntu machines to complete successfully but causes the CentOS machine to fail with an "Incorrect sudo password" error. This behavior is expected when different managed hosts use different privilege escalation passwords.

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass site.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.101]
ok: [192.168.56.102]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.101]
ok: [192.168.56.104]
ok: [192.168.56.102]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]
skipping: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0
   rescued=0    ignored=0
192.168.56.101   : ok=2    changed=0    unreachable=0    failed=0    skipped=1
   rescued=0    ignored=0
192.168.56.102   : ok=2    changed=0    unreachable=0    failed=0    skipped=1
   rescued=0    ignored=0
192.168.56.104   : ok=2    changed=0    unreachable=0    failed=0    skipped=1
   rescued=0    ignored=0
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat site.yml
---
- hosts: all
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"
```

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```
[web_servers]
192.168.56.120
192.168.56.121

[db_servers]
192.168.56.122

[file_servers]
192.168.56.123
```

Make sure to save the file and exit.

```
GNU nano 7.2                                inventory *
[web_servers]
192.168.56.101 ansible_user=serv1javillonar
192.168.56.102 ansible_user=serv2javillonar
192.168.1.23  ansible_user=cjavillonar

[db_servers]
192.168.56.104 ansible_user=javillonar
192.168.1.23  ansible_user=cjavillonar

[file_servers]
192.168.56.104 ansible_user=javillonar
```

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the site.yml by following the image below:

```
---
- hosts: all
  become: true
  pre_tasks:
    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"
    - name: install updates (Ubuntu)
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
  tasks:
    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"
    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```

```
GNU nano 7.2                                site.yml *
--
- hosts: all
  become: true

  pre_tasks:

    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      apt:
        upgrade: dist
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true

  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php

        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"
```

The pre-tasks command tells the ansible to run it before any other thing. In the pre-tasks, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at web_servers. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass site.yml
BECOME password:

PLAY [all] *****
*****

TASK [Gathering Facts] *****
*****
ok: [192.168.56.104]
ok: [192.168.56.101]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}
ok: [192.168.56.102]

TASK [install updates (CentOS)] *****
*****
skipping: [192.168.56.101]
ok: [192.168.56.102]

TASK [install updates (CentOS)] *****
*****
skipping: [192.168.56.101]
skipping: [192.168.56.102]
skipping: [192.168.56.104]

TASK [install updates (Ubuntu)] *****
*****
ok: [192.168.56.101]
ok: [192.168.56.102]
ok: [192.168.56.104]

PLAY [web_servers] *****
*****

TASK [Gathering Facts] *****
*****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for Ubuntu servers] *****
*****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [install apache and php for CentOS servers] *****
*****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.102   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104   : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

Run the site.yml file and describe the result.

- The playbook was executed successfully on the Ubuntu servers (Server 1, Server 2, and Server 3). Apache and PHP were already installed, so no changes were made. The CentOS server failed because a different sudo password was entered. This is expected since Ubuntu and CentOS use different sudo passwords.
4. Let's try to edit again the site.yml file. This time, we are going to add plays targeting the other servers. This time we target the db_servers by adding it on the current site.yml. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    yum:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb package (Ubuntu)
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"
```

(bottom part)

```
- hosts: db_servers
  become: true

  tasks:

  - name: install mariadb package (CentOS)
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: MariaDB-Restarting/Enabling
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb package (Ubuntu)
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"
```

Run the site.yml file and describe the result

- The database play successfully installed MariaDB on Server 3 (192.168.56.104). The CentOS installation task was skipped because Server 3 is an Ubuntu machine, while the Ubuntu MariaDB installation task was executed successfully. The MariaDB service was then successfully restarted and enabled on Server 3. The CentOS server still failed because its sudo password is different from the Ubuntu servers.

```
TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.104]

TASK [install mariadb package (Ubuntu)] *****
changed: [192.168.56.104]

TASK [MariaDB-Restarting/Enabling] *****
changed: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.102   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104   : ok=5    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: `systemctl status mariadb`. Do this on the CentOS server also.

```
javillonar@serv3javi:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.14 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-02 18:49:26 UTC; 1min 15s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
  Main PID: 21127 (mariabdb)
    Status: "Taking your SQL requests now..."
     Tasks: 10 (limit: 22793)
  Memory: 101.8M (peak: 102.2M)
       CPU: 738ms
    CGroup: /system.slice/mariadb.service
            └─21127 /usr/sbin/mariabdb
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ssh cjavillonar@192.168.1.23
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Thu Sep  3 03:13:20 2026 from 192.168.1.152
[cjavillonar@localhost ~]$ sudo systemctl status mariadb
[sudo] password for cjavillonar:
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-03 03:20:41 PST; 57s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
  Process: 8165 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
  Process: 8187 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
  Process: 8286 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
 Main PID: 8271 (mariabdd)
    Status: "Taking your SQL requests now..."
      Tasks: 14 (limit: 10500)
     Memory: 91.8M (peak: 120.8M)
        CPU: 642ms
    CGroup: /system.slice/mariadb.service
            └─8271 /usr/libexec/mariabdd --basedir=/usr

Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: The second is mysql@localhost, it has no password ei>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: you need to be the system 'mysql' user to connect.
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: After connecting you can set the password, if you wo>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: able to connect as any of these users with a passwor>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: See the MariaDB Knowledgebase at https://mariadb.com>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: Please report any problems at https://mariadb.org/ji>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: The latest information about MariaDB is available at>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: Consider joining MariaDB's strong and vibrant commu>
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: https://mariadb.org/get-involved/
Sep 03 03:20:41 localhost.localdomain systemd[1]: Started MariaDB 10.5 database server.
```

Describe the output

- The MariaDB service was verified using the systemctl status command. The output showed that the MariaDB service was active and running, confirming that the database server was installed and configured successfully.

6. Edit the site.yml again. This time we will append the code to configure installation on the file_servers group. We can add the following on our file.

```
- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

```
- hosts: file_servers
  become: true

  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.

Run the site.yml file and describe the result.

- The playbook successfully executed the db_servers and file_servers plays. Server 3 (192.168.56.104) was correctly identified as an Ubuntu database server, so the CentOS MariaDB task was skipped, while the Ubuntu MariaDB task completed successfully and the MariaDB service was restarted and enabled. The Samba package was also successfully installed on Server 3. The CentOS server still shows failed=1 because of its different sudo password, while Server 3 completed successfully with failed=0.

```
PLAY [db_servers] *****
TASK [Gathering Facts] *****
ok: [192.168.56.104]
TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.104]
TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.104]
TASK [MariaDB-Restarting/Enabling] *****
changed: [192.168.56.104]
PLAY [file_servers] *****
TASK [Gathering Facts] *****
ok: [192.168.56.104]
TASK [install samba package] *****
changed: [192.168.56.104]
PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.102   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104   : ok=7    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

The testing of the file_servers is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the site.yml file. Add tags to the playbook. After the name, we can place the tags: name_of_tag. This is an arbitrary command, which means you can use any name for a tag

```

- - -
- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    tags: always
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

```

```

- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    tags: apache,apache2,ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: apache,centos,httpd
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"

```

```

- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb package (Ubuntu)
    tags: db, mariadb,ubuntu
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"

- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    tags: samba
    package:
      name: samba
      state: latest

```

GNU nano 7.2

site.yml *

```
---
- hosts: all
  become: true

  pre_tasks:
    - name: install updates (CentOS)
      tags:
        - always
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      tags:
        - always
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true

  tasks:
    - name: install apache and php for Ubuntu servers
      tags:
        - apache
```

GNU nano 7.2

site.yml *

```
  name:
    - apache2
    - libapache2-mod-php
  state: latest
when: ansible_distribution == "Ubuntu"

- name: install apache and php for CentOS servers
  tags:
    - apache
    - centos
    - httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- hosts: db_servers
  become: true

  tasks:
    - name: install mariadb package (CentOS)
      tags:
        - centos
        - db
        - mariadb
      dnf:
        name: mariadb-server
```

GNU nano 7.2

site.yml *

```
when: ansible_distribution == "CentOS"

- name: install mariadb package (Ubuntu)
  tags:
    - db
    - mariadb
    - ubuntu
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: MariaDB-Restarting/Enabling
  tags:
    - db
    - mariadb
  service:
    name: mariadb
    state: restarted
    enabled: true

- hosts: file_servers
  become: true

  tasks:

    - name: install samba package
      tags:
        - samba
      package:
```

Make sure to save the file and exit.

Run the site.yml file and describe the result

-

```
wdjavillonar@workstation:~/CPE232_Javillonar$ nano site.yml
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass site.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.104]
ok: [192.168.56.102]
Fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [install updates (CentOS)] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]
skipping: [192.168.56.104]

TASK [install updates (Ubuntu)] *****
ok: [192.168.56.101]
ok: [192.168.56.102]
ok: [192.168.56.104]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for CentOS servers] *****

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.104]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.104]

TASK [MariaDB-Restarting/Enabling] *****
changed: [192.168.56.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install samba package] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.102   : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104   : ok=7    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

- The command displayed all available tags defined in the playbook. These tags can be used to execute only specific tasks instead of running the entire playbook.

2. On the local machine, try to issue the following commands and describe each result:

2.1 ansible-playbook --list-tags site.yml

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --list-tags site.yml

playbook: site.yml

  play #1 (all): all    TAGS: []
    TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
    TASK TAGS: [apache, centos, httpd, ubuntu]

  play #3 (db_servers): db_servers    TAGS: []
    TASK TAGS: [centos, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers TAGS: []
    TASK TAGS: [samba]
```

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

(i used my password in my CentOS)

```
wdjavillonar@workstation:~/CPE232_Javillonar$ nano site.yml
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --tags centos --ask-become-pass site.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.104]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]

TASK [install apache and php for CentOS servers] *****
ok: [192.168.1.23]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]

TASK [install mariadb package (CentOS)] *****
ok: [192.168.1.23]

PLAY [file_servers] *****

PLAY RECAP *****
192.168.1.23: ok=5  changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

2.3 ansible-playbook --tags db --ask-become-pass site.yml

(password from Ubuntu)

```
TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]
ok: [192.168.56.104]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.104]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.104]

TASK [MariaDB-Restarting/Enabling] *****
changed: [192.168.56.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

```
TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.104]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}
ok: [192.168.56.102]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.102   : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.104   : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

wdjavillonar@workstation:~/CPE232_Javillonar$
```

2.5 ansible-playbook --tags "apache,db" --ask-become-pass site.yml

```
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.104]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.104]

TASK [MariaDB-Restarting/Enabling] *****
changed: [192.168.56.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.102   : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.104   : ok=5    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

wdjavillonar@workstation:~/CPE232_Javillonar$
```

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server

```
- name: install apache and php for CentOS servers
  tags: apache,centos,httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

```
GNU nano 7.2 site.yml *
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
    when: ansible_distribution == "Ubuntu"

- name: install apache and php for CentOS servers
  tags:
    - apache
    - centos
    - httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
    when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags:
    - apache
    - centos
    - httpd
  service:
    name: httpd
    state: started
    enabled: true
    when: ansible_distribution == "CentOS"

- hosts: db_servers
  become: true
  
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Figure 3.1.1

You would also notice from our previous activity that we already created a module that runs a service.

```
- hosts: db_servers
  become: true
  tasks:

- name: install mariadb package (CentOS)
  tags: centos, db, mariadb
  dnf:
    name: mariadb-server
    state: latest
    when: ansible_distribution == "CentOS"

- name: "Mariadb- Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true
  
```

Figure 3.1.2

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

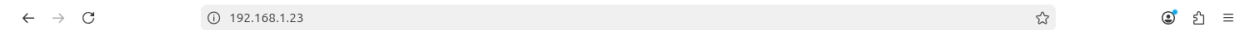
2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command `sudo systemctl stop httpd`. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd

service already.

```
wdjavillonar@workstation:~/CPE232_Javillonar$ sudo systemctl stop httpd
[sudo] password for wdjavillonar:
Sorry, try again.
[sudo] password for wdjavillonar:
Failed to stop httpd.service: Unit httpd.service not loaded.
wdjavillonar@workstation:~/CPE232_Javillonar$ ^C
wdjavillonar@workstation:~/CPE232_Javillonar$ ^C
wdjavillonar@workstation:~/CPE232_Javillonar$ ssh cjavillonar@192.168.1.23
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Thu Sep  3 03:53:38 2026 from 192.168.1.152
[cjavillonar@localhost ~]$ sudo systemctl stop httpd
[sudo] password for cjavillonar:
Sorry, try again.
[sudo] password for cjavillonar:
[cjavillonar@localhost ~]$ sudo systemctl status httpd
○ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: inactive (dead)
   Docs: man:httpd.service(8)
[cjavillonar@localhost ~]$
```

3. Go to the local machine and this time, run the site.yml file. Then after running the file, go again to the CentOS server and enter its IP address on the browser.



Unable to connect

Firefox can't establish a connection to the server at 192.168.1.23.

- The site could be temporarily unavailable or too busy. Try again in a few moments.
- If you are unable to load any pages, check your computer's network connection.
- If your computer or network is protected by a firewall or proxy, make sure that Firefox is permitted to access the web.

Try Again

```
PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]

TASK [install apache and php for Ubuntu servers] *****
skipping: [192.168.1.23]

TASK [install apache and php for CentOS servers] *****
ok: [192.168.1.23]

TASK [start httpd (CentOS)] *****
changed: [192.168.1.23]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]

TASK [install mariadb package (CentOS)] *****
ok: [192.168.1.23]

PLAY [file_servers] *****

PLAY RECAP *****
192.168.1.23      : ok=6    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.181   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.182   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.184   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0

wdjavillonar@workstation:~/CPE232_Javillonar$ 
wdjavillonar@workstation:~/CPE232_Javillonar$ sudo systemctl status httpd
Unit httpd.service could not be found.
wdjavillonar@workstation:~/CPE232_Javillonar$ ssh cjavillonar@192.168.1.23
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Thu Sep  3 04:05:26 2026 from 192.168.1.152
[cjavillonar@localhost ~]$ sudo systemctl status httpd
[sudo] password for cjavillonar:
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Thu 2026-09-03 04:05:23 PST; 3min 27s ago
     Docs: man:httpd.service(8)
  Main PID: 15544 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec:  0 B/sec"
    Tasks: 177 (limit: 10500)
   Memory: 18.8M (peak: 19.3M)
      CPU: 384ms
    CGroup: /system.slice/httpd.service
            └─15544 /usr/sbin/httpd -DFOREGROUND
               15551 /usr/sbin/httpd -DFOREGROUND
               15552 /usr/sbin/httpd -DFOREGROUND
               15553 /usr/sbin/httpd -DFOREGROUND
               15554 /usr/sbin/httpd -DFOREGROUND

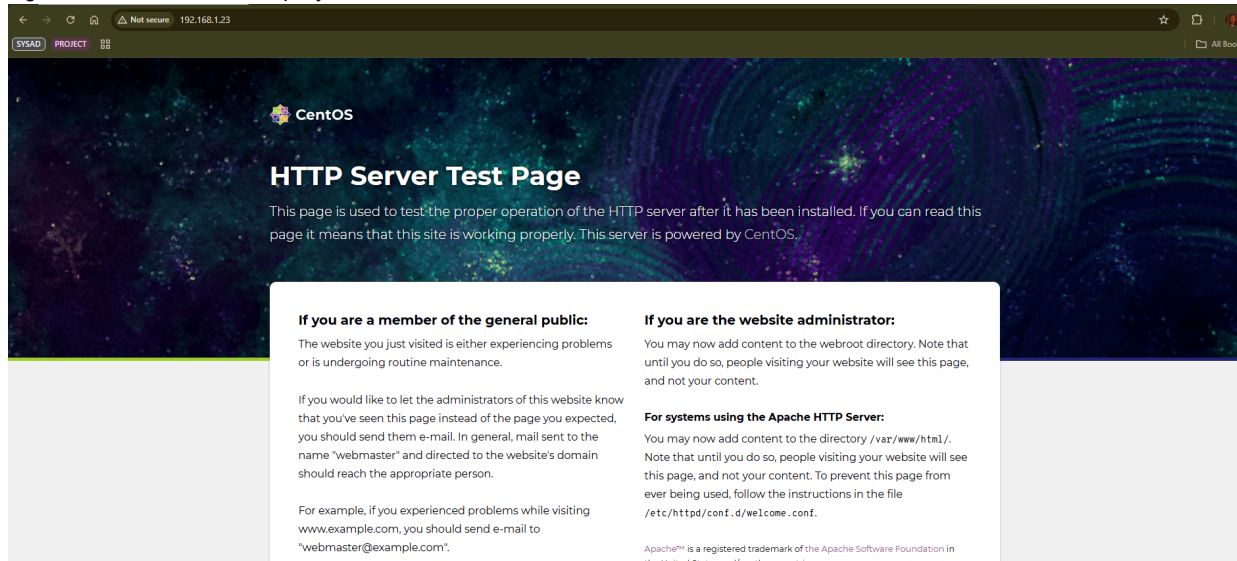
Sep 03 04:05:23 localhost.localdomain systemd[1]: Starting The Apache HTTP Server...
Sep 03 04:05:23 localhost.localdomain httpd[15544]: AH00558: httpd: Could not reliably determine the server's fully q
Sep 03 04:05:23 localhost.localdomain systemd[1]: Started The Apache HTTP Server.
Sep 03 04:05:23 localhost.localdomain httpd[15544]: Server configured, listening on: port 80
lines 1-22/22 (END)
```

Describe the result.

- The playbook was executed to start the Apache service automatically. Ansible connected to the target server and applied the service management task defined in the playbook.

To automatically enable the service every time we run the playbook, use the command `enabled: true` similar to

Figure 7.1.2 and save the playbook.



- After Ansible started the httpd service, the Apache webpage became accessible again through the CentOS server's IP address. The successful loading of the CentOS HTTP Server Test Page confirmed that the web service was running properly.

```
[cjavillonar@localhost ~]$ sudo firewall-cmd --list-services
cockpit dhcpv6-client http ssh
[cjavillonar@localhost ~]$ sudo systemctl is-enabled httpd
enabled
[cjavillonar@localhost ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Thu 2026-09-03 04:05:23 PST; 18min ago
     Docs: man:httpd.service(8)
  Main PID: 15544 (httpd)
    Status: "Total requests: 7; Idle/Busy workers 100/0;Requests/sec: 0.00629; Bytes served/sec: 12KB/sec"
     Tasks: 177 (limit: 10500)
    Memory: 22.0M (peak: 24.6M)
       CPU: 1.488s
    CGroup: /system.slice/httpd.service
            └─15544 /usr/sbin/httpd -DFOREGROUND
              └─15551 /usr/sbin/httpd -DFOREGROUND
                └─15552 /usr/sbin/httpd -DFOREGROUND
                  └─15553 /usr/sbin/httpd -DFOREGROUND
                    └─15554 /usr/sbin/httpd -DFOREGROUND

Sep 03 04:05:23 localhost.localdomain systemd[1]: Starting The Apache HTTP Server...
Sep 03 04:05:23 localhost.localdomain httpd[15544]: AH00558: httpd: Could not reliably determine the server's fully q
Sep 03 04:05:23 localhost.localdomain systemd[1]: Started The Apache HTTP Server.
Sep 03 04:05:23 localhost.localdomain httpd[15544]: Server configured, listening on: port 80
lines 1-22/22 (END)
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

1. Creating a Role for Web Servers

```
wdjavillonar@workstation:~/CPE232_Javillonar$ mkdir -p roles/web/tasks
wdjavillonar@workstation:~/CPE232_Javillonar$ nano roles/web/tasks/main.yml
```

```
GNU nano 7.2 roles/web/tasks/main.yml *
---
- name: Install Apache on Ubuntu
  apt:
    name: apache2
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: Install Apache on CentOS
  dnf:
    name: httpd
    state: latest
  when: ansible_distribution == "CentOS"
```

2. Creating a Role for Web Servers

```
GNU nano 7.2 roles/web/tasks/main.yml *
---
- name: Install Apache on Ubuntu
  apt:
    name: apache2
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: Install Apache on CentOS
  dnf:
    name: httpd
    state: latest
  when: ansible_distribution == "CentOS"

- name: Start Apache on CentOS
  service:
    name: httpd
    state: started
    enabled: true
  when: ansible_distribution == "CentOS"
```

3. Creating a Role for Database Servers

```
wdjavillonar@workstation:~/CPE232_Javillonar$ mkdir -p roles/db/tasks
wdjavillonar@workstation:~/CPE232_Javillonar$ nano roles/db/tasks/main.yml
```

```
GNU nano 7.2 roles/db/tasks/main.yml *
--
- name: Install MariaDB on Ubuntu
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: Install MariaDB on CentOS
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

- name: Start MariaDB
  service:
    name: mariadb
    state: started
    enabled: true
```

4. Combining Multiple Roles in a Playbook

```
GNU nano 7.2 roles.yml *
--
- hosts: web_servers
  become: true
  roles:
    - role: web
      tags:
        - web

- hosts: db_servers
  become: true
  roles:
    - role: db
      tags:
        - db
```

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Web Role

(ubuntu password)

```
wdjavillonar@workstation:~/CPE232_Javillonar$ nano roles.yml
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --tags web --ask-become-pass roles.yml
BECOME password:

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [web : Install Apache on Ubuntu] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [web : Install Apache on CentOS] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

TASK [web : Start Apache on CentOS] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.102   : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104   : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

(centos password)

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --tags web --ask-become-pass roles.yml
BECOME password:

PLAY [web_servers] *****

TASK [Gathering Facts] *****
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
ok: [192.168.1.23]
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [web : Install Apache on Ubuntu] *****
skipping: [192.168.1.23]

TASK [web : Install Apache on CentOS] *****
ok: [192.168.1.23]

TASK [web : Start Apache on CentOS] *****
ok: [192.168.1.23]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.104]: FAILED! => {"msg": "Incorrect sudo password"}

PLAY RECAP *****
192.168.1.23      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.101   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.102   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.104   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?
 - Putting servers into groups makes it easier to manage machines with the same purpose. We can run specific tasks only on web servers, database servers, or file servers.
2. What is the importance of tags in playbooks?
 - Tags allow us to run only the tasks we need instead of running the whole playbook. This saves time and makes it easier to control specific tasks.
3. Why do think some services need to be managed automatically in playbooks?
 - Some services need to run all the time for the server to work properly. Managing them automatically makes sure they are started and enabled without having to configure each server manually.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



```
[cjavillonar@localhost ~]$ sudo systemctl status mariadb
[sudo] password for cjavillonar:
Unit mariadb.service could not be found.
[cjavillonar@localhost ~]$ sudo systemctl status mariadb
Unit mariadb.service could not be found.
[cjavillonar@localhost ~]$ sudo dnf install mariadb-server -y
[sudo] password for cjavillonar:
Sorry, try again.
[sudo] password for cjavillonar:
CentOS Stream 9 - BaseOS                               5.7 kB/s | 4.3 kB    00:00
CentOS Stream 9 - BaseOS                               ] 7.3 kB/s |   0 B    00:00 ET
```

```
Installing      : perl-DBD-MariaDB-1.21-17.el9.x86_64      5/
Installing      : mariadb-errmsg-3:10.5.29-5.el9.x86_64    6/
Installing      : perl-File-Copy-2.34-483.el9.noarch       7/
Running scriptlet: mysql-selinux-1.0.18-1.el9.noarch      8/
Installing      : mysql-selinux-1.0.18-1.el9.noarch      8/
Running scriptlet: mysql-selinux-1.0.18-1.el9.noarch      8/
libsemanage.semanage_direct_install_info: Overriding mysql module at lower priority 100 with module at priority 200.

Installing      : mariadb-backup-3:10.5.29-5.el9.x86_64    9/
Installing      : mariadb-gssapi-server-3:10.5.29-5.el9.x86_64 10/
Installing      : mariadb-server-utils-3:10.5.29-5.el9.x86_64 11/
Running scriptlet: mariadb-server-3:10.5.29-5.el9.x86_64 12/
Installing      : mariadb-server-3:10.5.29-5.el9.x86_64 12/
Running scriptlet: mariadb-server-3:10.5.29-5.el9.x86_64 12/
Installing      : mariadb-3:10.5.29-5.el9.x86_64          13/
Running scriptlet: mysql-selinux-1.0.18-1.el9.noarch      13/
Running scriptlet: mariadb-3:10.5.29-5.el9.x86_64        13/
Verifying       : mariadb-3:10.5.29-5.el9.x86_64         1/
Verifying       : mariadb-backup-3:10.5.29-5.el9.x86_64  2/
Verifying       : mariadb-common-3:10.5.29-5.el9.x86_64  3/
Verifying       : mariadb-connector-c-3.2.6-1.el9.x86_64  4/
Verifying       : mariadb-connector-c-config-3.2.6-1.el9.noarch 5/
Verifying       : mariadb-errmsg-3:10.5.29-5.el9.x86_64  6/
Verifying       : mariadb-gssapi-server-3:10.5.29-5.el9.x86_64 7/
Verifying       : mariadb-server-3:10.5.29-5.el9.x86_64  8/
Verifying       : mariadb-server-utils-3:10.5.29-5.el9.x86_64 9/
Verifying       : mysql-selinux-1.0.18-1.el9.noarch      10/
Verifying       : perl-DBD-MariaDB-1.21-17.el9.x86_64    11/
Verifying       : perl-File-Copy-2.34-483.el9.noarch     12/
Verifying       : perl-Sys-Hostname-1.23-483.el9.x86_64  13/

Installed:
mariadb-3:10.5.29-5.el9.x86_64      mariadb-backup-3:10.5.29-5.el9.x86_64      mariadb-common-3:10.5.29-5.el9.x86_64
mariadb-connector-c-3.2.6-1.el9.x86_64 mariadb-connector-c-config-3.2.6-1.el9.noarch mariadb-errmsg-3:10.5.29-5.el9.x86_64
mariadb-gssapi-server-3:10.5.29-5.el9.x86_64 mariadb-server-3:10.5.29-5.el9.x86_64 mariadb-server-utils-3:10.5.29-5.el9.x86_64
mysql-selinux-1.0.18-1.el9.noarch      perl-DBD-MariaDB-1.21-17.el9.x86_64      perl-File-Copy-2.34-483.el9.noarch
perl-Sys-Hostname-1.23-483.el9.x86_64

Complete!
[cjavillonar@localhost ~]$ rpm -q mariadb-server
mariadb-server-10.5.29-5.el9.x86_64
[cjavillonar@localhost ~]$
```



COMPUTER
ENGINEERING

QUEZON CITY

```
[cjavillonar@localhost ~]$ sudo systemctl status mariadb
o mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
[cjavillonar@localhost ~]$ sudo systemctl enable mariadb
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[cjavillonar@localhost ~]$ sudo systemctl start mariadb
[cjavillonar@localhost ~]$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-03 03:20:41 PST; 2s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 8165 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
   Process: 8187 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
   Process: 8286 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
  Main PID: 8271 (mariadb)
    Status: "Taking your SQL requests now..."
     Tasks: 14 (limit: 10500)
    Memory: 91.8M (peak: 120.8M)
       CPU: 626ms
    CGroup: /system.slice/mariadb.service
            └─8271 /usr/libexec/mariadb --basedir=/usr

Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: The second is mysql@localhost, it has no password either, but
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: you need to be the system 'mysql' user to connect.
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: After connecting you can set the password, if you would need to be
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: able to connect as any of these users with a password and without sudo
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: Please report any problems at https://mariadb.org/jira
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: The latest information about MariaDB is available at https://mariadb.org/.
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: Consider joining MariaDB's strong and vibrant community:
Sep 03 03:20:41 localhost.localdomain mariadb-prepare-db-dir[8226]: https://mariadb.org/get-involved/
Sep 03 03:20:41 localhost.localdomain systemd[1]: Started MariaDB 10.5 database server.
```